

PYTHON O CÓDIGO DA
RESISTÊNCIA CONTRA O IMPÉRIO

PYTHON: ARQUIVOS E CRIPTOGRAFIA



7

LISTA DE FIGURAS

Figura 1 – Resultado da leitura de um arquivo com readlines()	11
---	----

EXEMPLO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Iniciando a prática com arquivos.....	6
Código-fonte 2 – Criar arquivo	7
Código-fonte 3 – Sobrescrita de arquivo	8
Código-fonte 4 – Gerando um código HTML.....	9
Código-fonte 5 – Leitura de um arquivo texto.....	11
Código-fonte 6 – Exemplo para gerenciamento de inventário.....	13
Código-fonte 7 – Funções para manipulação do inventário	15
Código-fonte 8 – Estrutura para chamar as funções de manipulação do inventário ..	16
Código-fonte 9 – Realizando slice de string	18
Código-fonte 10 – Utilizando o método find()	19
Código-fonte 11 – Desmembrando uma string.....	20
Código-fonte 12 – Desmembrando uma string com método Split().....	23
Código-fonte 13 – Criptografia com DES	25
Código-fonte 14 – Criptografia com AES	26
Código-fonte 15 – Conversão Base64	26

SUMÁRIO

1 INTRODUÇÃO	5
2 MANIPULAÇÃO DE ARQUIVOS	5
2.1 Criando um arquivo e adicionando conteúdos	6
2.2 Realizando a leitura do arquivo	10
3 GERANDO UMA ROTINA PARA INVENTÁRIO	12
4 MANIPULAÇÃO DE STRINGS NA RECUPERAÇÃO DOS DADOS DE ARQUIVOS	17
4.1 Slice de String	18
4.2 Método find() e o poderoso método split()	19
5 TECNOLOGIA LIGADA À SEGURANÇA DE INFORMAÇÃO.....	23
5.1 Criptografia.....	23
5.2 Data Encryption Standard (DES).....	24
5.3 <i>Advanced Encryption Standard</i> (AES)	25
6 CONVERSÃO BASE64	26
7 SIMULANDO UM ATAQUE RANSOMWARE	27
7.1 Finalizando o capítulo.....	27
REFERÊNCIAS.....	28

1 INTRODUÇÃO

Até o momento, nossos dados só existiam enquanto o sistema estivesse aberto e/ou a sua estrutura não fosse limpa ou sobrescrita. A partir de agora, nossos dados passam a ser mais valorizados pelas nossas rotinas e, então, persistiremos esses dados em disco. Isso significa que eles passarão a existir fisicamente em uma estrutura de arquivo, o que permitirá que você transporte os dados coletados de uma máquina para outra ou até mesmo atualize um sistema por meio das instruções encontradas em arquivos gerados por outros.

Também será possível gerar um arquivo de *script* e atualizar a configuração de equipamentos como roteadores e switches, avaliar arquivos de log a fim de encontrar informações sobre comportamentos não autorizados de usuários, realizar auditorias, entre muitas outras possibilidades. Serão apresentadas ainda algumas funções para a manipulação de strings de suma importância também para a manipulação de um arquivo já gerado. Além disso, será apresentado formas de criptografia que permitem garantir a segurança dos arquivos de uma máquina, inclusive, servem como base para alguns tipos de ataque como *ransomware*.

2 MANIPULAÇÃO DE ARQUIVOS

Para a manipulação dos arquivos, utilizaremos uma função denominada `open()`, que permite várias ações:

- **`open("<caminhoDoArquivo><nomeDoArquivo>", "w")`** => indica que você está abrindo um arquivo para o modo de escrita (`w` => `write`), ou seja, permite que você escreva nele, caso o arquivo já exista, ele será sobrescrito. É como se você adquirisse um novo "caderno", completamente em branco.
- **`open("<caminhoDoArquivo><nomeDoArquivo>", "r")`** => com a letra `"r"` (`read`) no segundo parâmetro da função, você abrirá o arquivo somente em modo de leitura, isso permite que outra pessoa, em outro computador, possa abrir esse arquivo para edição, mas você poderá apenas "consumir" os dados que estiverem dentro desse arquivo. Nesse caso, podemos fazer a analogia com um livro, isto é, você não o preencherá, tão pouco o alterará, apenas fará a leitura.

- **open("<caminhoDoArquivo><nomeDoArquivo>", "a")** => dessa forma, você poderá ler e escrever no arquivo especificado como se fosse um diário, no qual você acrescentará conteúdos de acordo com algum evento, periodicidade ou qualquer outro advento do dia a dia, mas sempre no final do diário, em outros termos a principal ideia é seguir concatenando os dados, por isso, a letra "a" referindo-se à *"append"* (anexar). Um exemplo bem prático para isso seria o controle de um arquivo de log, tudo o que for adicionado será acrescentado ao final dos dados que já existem.
- **open("<caminhoDoArquivo><nomeDoArquivo>", "x")** => permite criar um arquivo em modo exclusivo (*eXclusive*), uma vez que você criou/abriu o arquivo, ninguém mais poderá abri-lo. Caso você tente abrir um arquivo que já existe, será retornada uma falha.
- **open("<caminhoDoArquivo><nomeDoArquivo>", "t")** => o arquivo que for aberto com o parâmetro "t" (text) retornará para o Python o seu conteúdo como *string*, diferentemente do parâmetro "b", que retornaria os dados em formato binário e exigiria uma conversão para *string*, caso fosse necessário.

Você pode combinar os modos (texto ou binário) com as ações (exclusive, append, write ou read, utilizando ou não o operador "+", por exemplo: "w+b" ou "wb".

2.1 Criando um arquivo e adicionando conteúdos

Iniciaremos com a criação de uma rotina que gerará um arquivo, veremos alguns procedimentos simples e, então, ao término do capítulo, nos aprofundaremos um pouco mais.

Vamos à nossa estrutura, ou seja, dentro do VS Code crie um diretório "Capítulo5" e um novo arquivo "**GravarArquivo.py**".

Nesse arquivo, montaremos o código abaixo. Quando executá-lo, ele não funcionará, explicaremos no parágrafo seguinte:

```
with open("teste.txt", "r") as arquivo:
    conteudo = arquivo.read()
```

Código-fonte 1 – Iniciando a prática com arquivos
Fonte: Elaborado pelo autor (2020)

Para um primeiro teste, apresentamos a utilização da função **open()** com o comando “*with*”, esse comando permitirá representar, dentro do bloco indentado, o arquivo “teste.txt” por meio do “*alias*” *arquivo*. Outra grande vantagem que obteremos todas as vezes que utilizarmos a função **open()** combinada ao comando *with* é que o controle do encerramento do arquivo em memória ficará por conta do comando *with*, o que fará com que você não precise utilizar o método **close()**, tampouco o arquivo ficará aberto na memória sem qualquer necessidade. Por isso, recomendo que utilize esse combinado **open() + with**. Você não vai se arrepender!

Seguindo a leitura do código já apresentado, perceba que colocamos, no primeiro parâmetro da função **open()**, o arquivo “teste.txt” sem qualquer caminho sendo especificado. Isso indica que o Python considerará o caminho do seu arquivo que contém o código-fonte e que, conseqüentemente, estará sendo executado, ou seja, o caminho adotado para o arquivo “teste.txt” será o mesmo do arquivo “GravarArquivo.py”.

Caso queira, nada impede de definir um caminho para o arquivo. Você deverá substituir “teste.txt” por, por exemplo, “c:\meus_arquivos\teste.txt”. Perceba que o destaque negrito representa o caminho, isso significa que terá que ter acesso ao “c:” e que, dentro desta unidade, deverá existir uma pasta chamada “meus_arquivos”, uma vez que o método **open()** cria arquivos, não diretórios.

No segundo parâmetro, estamos especificando o valor “r”, que significa que abriremos o arquivo somente para leitura e, dentro do *with*, chamamos a função **read()**, que lê o conteúdo do arquivo. Mas como você pode observar, foi gerado um erro ao tentar executar essas duas linhas de código. Isso aconteceu porque o arquivo não existe. Como você pode tentar efetuar a leitura de um arquivo sem que ele exista? O Python, então, retornará a mensagem de erro: **FileNotFoundError: [Errno 2] No such file or directory: 'teste.txt'**. Isso foi apenas para que você pudesse já observar o funcionamento do segundo parâmetro, uma vez que a ideia principal, nessa parte, é a de criar um arquivo e não a de ler o arquivo.

Vamos alterar o código para:

```
with open("teste.txt", "w") as arquivo:  
    arquivo.write("Nunca foi tão fácil criar um arquivo.")
```

Código-fonte 2 – Criar arquivo
Fonte: Elaborado pelo autor (2020)

Dois pontos a serem observados:

- Alteramos o segundo parâmetro de “r” para “w”, ou seja, agora o arquivo será aberto para escrita e não leitura.
- Dentro do **with**, não estamos mais utilizando o método **read()** e, sim, o método **write()**, que nos permite passar o conteúdo que queremos para dentro do arquivo.

Você deve estar se perguntando: mas, se o arquivo “teste.txt” não existe, o que ocorrerá? Execute o arquivo e veja... Agora funcionou, certo? Isso ocorreu devido à função do parâmetro “w”, que cria o arquivo, se você executar novamente, com outra mensagem, verá que o arquivo anterior foi sobrescrito e repare que o arquivo foi criado com o seu arquivo de código, conforme apresentado na imagem a seguir (a não ser que você tenha especificado um caminho diferente para o arquivo).

Repare que os acentos podem estar com caracteres estranhos, isso pode ou não acontecer, depende da configuração da sua IDE para a leitura de *encodes* para exibição de caracteres do tipo texto. Caso você tenha problemas com caracteres acentuados, isso pode ser removido utilizando o comando “control + ,” e, em seguida, buscar pela configuração “Files.Encoding” e alterá-la para **utf8**, **windows-1252** ou **ISO-8859-1** ou, ainda, algum outro *encoding* compatível com o seu layout de teclado, idioma e/ou sistema operacional.

Agora, perceba que ao colocarmos duas linhas com o método `write()`, o conteúdo das duas linhas será adicionado no arquivo, mas se utilizarmos novamente o método “**with**” com o mesmo arquivo, ele sobrescreverá o arquivo anterior. Teste o código a seguir e veja como ficará:

```
with open("teste.txt", "w") as arquivo:
    arquivo.write("Nunca foi tão fácil criar um arquivo.")

with open("teste.txt", "w") as arquivo:
    arquivo.write("\nContinuação do texto.")
```

Código-fonte 3 – Sobrescrita de arquivo
Fonte: Elaborado pelo autor (2020)

Esse código gerará um arquivo e escreverá: “Nunca foi tão fácil criar um arquivo”. O arquivo será encerrado, em seguida, aberto novamente e, então, a frase

“Continuação do texto” será escrita nele, mas, ao abri-lo, você encontrará apenas a segunda frase, isso porque o primeiro arquivo foi sobrescrito e não concatenado.

Essa é a função da opção “a”, que podemos utilizar no segundo parâmetro, altere somente no segundo *with* o “w” por “a” e veja como funcionará melhor agora. Vamos abrir um novo arquivo e gerar um *html* simples, para que você possa ter a noção exata do quão importante pode ser a geração de um arquivo dentro de uma linguagem de programação. Crie um arquivo e o nomeie “ArquivoHTML.py”, monte o seguinte código:

```
with open("pagina.html", "w") as pagina:
    pagina.write("<body> <h1> Este é um teste para página WEB  
</h1>")
    pagina.write("<br><h2> Abaixo seguem alguns nomes  
importantes para o projeto: </h2>")
    pagina.write("<h3>")
    nome=""
    while nome!="SAIR":
        nome = input("Digite um nome ou SAIR: ").upper()
        if nome!="SAIR":
            pagina.write("<br>"+nome)
    pagina.write("</h3></body>")
```

Código-fonte 4 – Gerando um código HTML
Fonte: Elaborado pelo autor (2020)

No código, utilizamos a criação do arquivo para gerarmos um arquivo *html*, ou seja, um website simples (afinal, a ideia aqui não é desenvolvê-los “*front end*”) para que veja quantas possibilidades é possível gerar com a criação de arquivos. Podendo ir desde gerar um simples arquivo texto, passando por uma página html até mesmo criar um código que geraria mais arquivos com códigos.

Somente para que entenda, de maneira simples, o HTML é desenvolvido entre *tags* (nome atribuído aos seus comandos). Essas *tags* são abertas como o <h1> e encerradas sempre com uma barra entre os sinais de menor e maior, por exemplo </h1>. Vamos às tags que utilizamos no código apresentado:

- <body>: indica o início do conteúdo que visualizará na página.
- <h1>: uma formatação maior para títulos e, à medida que muda o número, a fonte vai diminuindo, por exemplo, <h2> tem uma fonte menor que <h1>, entretanto, é maior que <h3>.

- `<br>`: é uma tag que não precisa ser encerrada, pois sua função é simplesmente dar uma quebra de linha.

Agora que já vimos as *tags*, vamos analisar o nosso código: criamos um arquivo chamado “pagina.html”, a extensão “html” é para que o arquivo seja interpretado como uma página pelo seu browser. Logo depois, abrimos o `<body>` e colocamos um título em `<h1>`. Em seguida, abrimos uma nova linha e digitamos um conteúdo com formatação `<h2>`. Abrimos o `<h3>`, criamos uma variável e montamos um laço que permitirá digitar vários dados nessa variável. Enquanto o seu conteúdo for diferente de “SAIR”, dentro do *while*, o conteúdo da variável será escrito, na página *html*, somente se não for igual a “SAIR”. Após o laço *while* encerrar, fecharemos a formatação `<h3>` e o `<body>`. Rode sua aplicação, digite alguns nomes conforme for solicitado e, após encerrar a aplicação digitando “SAIR”, você terá um arquivo chamado “pagina.html”. Para abri-lo no browser, clique sobre o arquivo com o botão direito do mouse, escolha a opção “Copy Path” e depois cole no browser que preferir.

Show! Se aprender um pouquinho mais sobre HTML, já poderá gerar relatórios ou dashboards poderosíssimos. Repare que, nesse exemplo, utilizamos apenas variáveis, imagine o que conseguirá fazer acrescentando as listas e os dicionários de dados. Pronto, já pode montar o seu diário de bordo!

2.2 Realizando a leitura do arquivo

Já vimos como criar os nossos arquivos, então, partiremos, agora, para a leitura do arquivo e isso será fundamental. É muito importante, em qualquer sistema computacional, o armazenamento dos dados, mas isso não traz nada de inovador e/ou agregador ao seu cliente, o diferencial está na leitura dos dados, que deverá trazer, para o seu cliente, dados na forma de informações, de uma maneira simples e confiável, pois, por meio delas, decisões gerenciais e/ou estratégicas serão tomadas.

Por isso, armazenar é importante, mas recuperar de maneira inteligente os dados é muito mais. Vamos para uma prática simples para esquentarmos, crie um arquivo de código chamado “LeituraArquivo.py” e digite o seguinte código:

```
with open("teste.txt", "r") as arquivo:
    conteudo=arquivo.read()
```

```
print("Tipo de dado da variável", type(conteudo))  
print("\nConteúdo do arquivo:\n", conteudo)
```

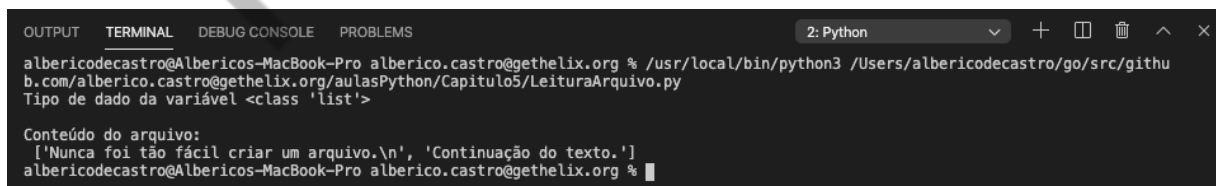
Código-fonte 5 – Leitura de um arquivo texto
Fonte: Elaborado pelo autor (2020)

Nesse código, abrimos o arquivo teste.txt para leitura ("r") e atribuímos, para a variável "conteudo", todo o conteúdo do arquivo (quando utilizamos o método **read()**), depois, imprimimos o tipo da variável e, na última linha, imprimimos o conteúdo do arquivo que está dentro da variável. O tipo de dado da variável "conteudo" que será exibido é "str", ou seja, string, isso porque, quando não definimos o modo de saída, o valor-padrão é o "t"(text), por isso "r" ou "rt" ou "r+t" como segundo parâmetro retornarão o mesmo tipo.

Podemos mudar para que a saída seja em bytes, para isso, substitua "r" por "rb" ou "r+b" e execute novamente esse bloco de código. Verá que não só o tipo mudará para "byte" como também o conteúdo do arquivo a ser exibido terá uma formatação diferente, por *byte*. O mais comum mesmo é fazê-lo utilizando a saída no formato texto, não é à toa que ele é a forma-padrão.

Outra alteração simples refere-se à forma como podemos retornar o conteúdo do arquivo. No código anterior, utilizamos a função **read()**, que retornará todo o conteúdo como se fosse uma única string. Façamos o seguinte teste: substitua **read()** por **readlines()** e execute novamente o seu código.

Percebeu a diferença? A saída da variável "conteudo" não é mais uma string e, sim, uma lista formada por duas posições, uma para cada linha, como podemos observar no resultado, que deve ser semelhante ao da seguinte figura:



```
albericodecastro@Albericos-MacBook-Pro alberico.castro@gethelix.org % /usr/local/bin/python3 /Users/albericodecastro/go/src/github.com/alberico.castro@gethelix.org/aulasPython/Capitulo5/LeituraArquivo.py  
Tipo de dado da variável <class 'list'>  
  
Conteúdo do arquivo:  
['Nunca foi tão fácil criar um arquivo.\n', 'Continuação do texto.']  
albericodecastro@Albericos-MacBook-Pro alberico.castro@gethelix.org %
```

Figura 1 – Resultado da leitura de um arquivo com **readlines()**

Fonte: Elaborada pelo autor (2020)

Será muito útil a função **readlines()** quando optarmos por quebrar um grande arquivo de texto em partes, para que possamos retirar somente os dados que nos

interessarem. Por falar nisso, vamos para um exemplo prático, no qual poderemos explorar mais ainda os conceitos, funções e métodos apresentados até o momento.

3 GERANDO UMA ROTINA PARA INVENTÁRIO

Uma das suas atribuições dentro do projeto será a de gerir todo o inventário de ativos que fazem parte da rede do seu cliente. É fundamental saber quantos são, quais são e onde estão distribuídos os ativos que fizerem parte da rede. Além disso, muitas outras informações podem ser devidamente documentadas, como licenças de softwares, controle sobre o que deve trafegar na rede ou em partes dela, local definido para armazenamento de dados específicos, controle de atualizações. Enfim, o gerenciamento e a segurança de uma rede, sem dúvida, começam com um inventário gerenciado, atualizado e funcional.

Montaremos uma rotina pequena e simples, a princípio, para que possamos praticar as técnicas para a manipulação de arquivos vistas até o momento, com uma aplicação prática que, certamente, fará parte das suas atribuições como um profissional de defesa cibernética. Vamos começar criando um arquivo chamado: “Inventario.py” e armazenaremos apenas os seguintes dados: número patrimonial do ativo, descrição do ativo, data da última atualização e nome do departamento em que está localizado.

Primeiro definiremos uma estrutura de dados para armazená-lo; eles serão recebidos pelo colaborador responsável por catalogar os ativos, e, então, persistiremos os dados para um arquivo, para que possam ser recuperados, “backupeados”, alterados, excluídos e estejam disponíveis para qualquer outra consulta que possa ser necessária posteriormente.

Utilizaremos a estrutura de um dicionário de dados para armazená-los enquanto estiverem na condição de dados voláteis, em que a chave será o número patrimonial do ativo (que não pode ser repetido). Sobre essa chave, teremos uma lista com o departamento, data da última alteração e descrição.

Após a gerência dos dados no dicionário, persistiremos em um arquivo do tipo “csv”, que representa um padrão de arquivo no qual os dados poderão ser abertos dentro do Excel e de outros programas que aceitam esse tipo de arquivo. Isso garantirá maior flexibilidade e portabilidade sobre os dados persistidos.

Agora, vamos montar o seguinte código:

```
inventario={}
opcao=int(input("Digite: "
    "\n<1> para registrar ativo"
    "\n<2> para persistir em arquivo"
    "\n<3> para exibir ativos armazenados: "))
while opcao>0 and opcao<4:
    if opcao==1:
        resp="S"
        while resp=="S":
            inventario[input("Digite o número patrimonial: ")] =[
                input("Digite a data da última atualização: "),
                input("Digite a descrição: "),
                input("Digite o departamento: ")]
            resp=input("Digite <S> para continuar.").upper()
        elif opcao==2:
            with open("inventario.csv", "a") as inv:
                for chave, valor in inventario.items():
                    inv.write(chave + ";" + valor[0] + ";" +
                        valor[1] + ";" +valor[2]+"\\n")
                print("Persistido com sucesso!")
        elif opcao==3:
            with open("inventario.csv", "r") as inv:
                print(inv.readlines())
    opcao=int(input("Digite: "
        "\n<1> para registrar ativo"
        "\n<2> para persistir em arquivo"
        "\n<3> para exibir ativos armazenados: "))
```

Código-fonte 6 – Exemplo para gerenciamento de inventário

Fonte: Elaborado pelo autor (2020)

Vejamos alguns pontos importantes para serem abordados:

- Criamos um dicionário de dados chamado “inventario” e montamos um menu de escolha, com as opções 1, 2 e 3. Enquanto o usuário digitar qualquer um dos três valores, o programa continuará sendo executado, entretanto, para qualquer outro valor, o programa será encerrado.
- Se o valor digitado for “1”, colocaremos o usuário em outro laço “while” e, enquanto ele digitar “S”, ele poderá adicionar itens para o nosso dicionário “inventario”.
- Se o valor digitado for “2”, abriremos o arquivo “inventario.csv” em modo de concatenação e, então, para cada objeto encontrado no nosso dicionário, adicionaremos uma linha no arquivo. Na linha, escreveremos a chave e os valores desta chave separados por “;” (para o Excel, isso indicará que cada

valor deverá ficar em uma coluna). Ao término da linha, adicionamos uma quebra de linha “\n”, isso também indicará ao Excel uma quebra de linha, indicando que temos, na sequência, outro objeto.

- Se, finalmente, o valor digitado for “3”, então, abriremos o arquivo em modo de leitura e utilizaremos o método “readlines()” para exibirmos todo o conteúdo do arquivo.

Após a execução do seu projeto e a verificação de que esteja tudo ok, dentro do próprio VS Code, abra o seu arquivo “csv”, assim como fizemos com o arquivo “txt”. Será aberto como um arquivo de texto normal, e para melhor visualizar os dados do arquivo csv, é preciso instalar um plugin, a própria IDE recomendará a instalação. Fica aqui sugerido o uso do “Rainbow CSV”, um plugin leve e que ajuda a distinguir as colunas de dados no arquivo csv.

É importante ficar atento ao uso de plug-ins para que você possa explorar todos os recursos adicionais que a IDE poderá lhe oferecer para a manipulação de arquivos “CSV”, nesse caso. Essa atualização pode ser necessária para outros conteúdos que veremos no decorrer do curso. Sempre que puder, realize-as para que tenha uma IDE com o máximo de recursos possíveis.

Procure abrir o seu arquivo no Excel, caso o tenha instalado em seu computador, você verá que os itens estarão devidamente separados e que você os poderá manipular, ordenar e realizar qualquer outra operação necessária com os dados que foram persistidos no arquivo CSV.

Perceba que o nosso código ficou um pouco extenso, e oferece poucas funções para o usuário. Isso significa que seria interessante, para esses exemplos, aplicarmos o conceito das funções e da modularização. Por isso, vamos criar as funções que simplificariam esse código.

Seguiremos aprimorando nossa estrutura de projeto e, em vez de criarmos simplesmente um arquivo com as funções, vamos criar um diretório chamado “Funcoes” e, dentro desse, criaremos todos os nossos arquivos que armazenarão funções. Começaremos criando o arquivo “Funcoes_Arquivos.py”, aproveite também para arrastar o arquivo “Funcoes.py” que estava dentro do diretório “Dicionarios” para o nosso pacote “Funcoes”.

Renomeie o arquivo “Funcoes.py” para “Funcoes_Dicionarios.py”, somente a fim de identificar sua origem. Para renomear, clique com o botão direito sobre o arquivo, escolha a opção “Rename”.

Pronto, dessa forma, todos os nossos arquivos que armazenarem funções estarão localizados em um mesmo pacote, o que nos facilitará reaproveitar funções entre projetos diferentes e, também, padronizar as importações. Caso queira utilizar arquivos com funções de diretórios superiores ou de mesmo nível, é necessário criar arquivos “__init.py__” em cada um dos diretórios, como o intuito de permitir que o python compreenda a estrutura de diretórios do projeto.

Agora, dentro do arquivo “Funcoes_Arquivos.py”, monte o seguinte código:

```
def chamarMenu():
    escolha = int(input("Digite: "
        "\n<1> para registrar ativo"
        "\n<2> para persistir em arquivo"
        "\n<3> para exibir ativos armazenados: "))
    return escolha

def registrar(dicionario):
    resp="S"
    while resp=="S":
        dicionario[input("Digite o número patrimonial: ")] =[
            input("Digite a data da última atualização: "),
            input("Digite a descrição: "),
            input("Digite o departamento: ")]
        resp=input("Digite <S> para continuar.").upper()

def persistir(dicionario):
    with open("inventario.csv", "a") as inv:
        for chave, valor in dicionario.items():
            inv.write(chave + ";" + valor[0] + ";" +
                valor[1] + ";" +valor[2]+"\\n")
    return "Persistido com sucesso"

def exibir():
    with open("inventario.csv", "r") as inv:
        linhas=inv.readlines()
    return linhas
```

Código-fonte 7 – Funções para manipulação do inventário

Fonte: Elaborado pelo autor (2020)

Foram criadas, no código apresentado, quatro funções:

- **chamarMenu():** apenas exibe para o usuário as opções que ele possui para a manipulação do inventário e retorna o valor selecionado por meio da variável “escolha”.
- **registrar():** receberá o dicionário responsável por armazenar os ativos e, então, o preencherá, enquanto o usuário digitar “S”.
- **persistir():** ficou encarregada de descarregar o dicionário que for passado como parâmetro para dentro do arquivo “inventario.csv”.
- **exibir():** retornará o conteúdo das linhas do arquivo “inventario.csv” por meio da variável “linhas”.

Agora, vamos atualizar o nosso arquivo “Inventario.py”, com o seguinte código:

```
from Funcoes.Funcoes_Arquivos import *
inventario={}
opcao=chamarMenu()
while opcao>0 and opcao<4:
    if opcao==1:
        registrar(inventario)
    elif opcao==2:
        persistir(inventario)
    elif opcao==3:
        print(exibir())
    opcao = chamarMenu()
```

Código-fonte 8 – Estrutura para chamar as funções de manipulação do inventário

Fonte: Elaborado pelo autor (2020)

O código do nosso arquivo “Inventario.py” tornou muito mais simples e muito mais fácil aplicar alterações e/ou manutenções. Uma atenção para a linha na qual chamamos o método `exibir()`, pois o chamamos dentro de um `print()` que será responsável por exibir tudo o que método `exibir()` retornar.

Interessante apontar que o arquivo “inventario.csv” é utilizado dentro das funções que estão no arquivo `Funcoes_Arquivo.py`, que, por sua vez, está no pacote `Funcoes`, já o arquivo “inventario.csv” está no pacote no diretório raiz, ou seja, estão em pacotes diferentes.

Mesmo assim, as funções reconhecem o arquivo “inventario.csv”, pois o caminho que utilizarão é o do qual forem “chamadas”, ou seja, do arquivo “Inventario.py”, que é o mesmo do arquivo “inventario.csv”.

Agora, repare onde será fundamental o programador Python... A saída dos dados do “inventario.csv”, pelo Python, está clara? Se tivermos 200 ativos cadastrados, imagine como ficaria a saída. Uma bagunça, não é mesmo? E se quiséssemos saber, por exemplo, quais as descrições dos ativos que pertencerem a um departamento em específico ou, ainda, quais ativos foram atualizados hoje. Agora, caberá a você recuperar esses dados e trabalhá-los da melhor forma possível, trazendo inteligência para o seu gerenciamento do ambiente. Será fundamental, neste momento, a manipulação de strings e é com esse assunto que trabalharemos no próximo tópico.

4 MANIPULAÇÃO DE STRINGS NA RECUPERAÇÃO DOS DADOS DE ARQUIVOS

No tópico anterior, paramos com a missão de melhorarmos a recuperação dos dados de um arquivo. Atualmente, o nosso arquivo “Inventario.py” retorna todas as informações do arquivo “inventario.csv” de uma forma nada agradável, por exemplo, para que se possa apresentar em uma reunião. Começaremos pelo seguinte: o número patrimonial do ativo, normalmente, não é um dado importante para ser exibido, salvo em consultas específicas.

É como se eu fosse até o supermercado e comprasse um refrigerante, mas, antes, quisesse saber o código que representa o refrigerante no estoque do supermercado. Insano, não é mesmo? Imagine-se, agora, fazendo as compras do mês e querendo saber todos os códigos de todos os produtos, ou seja, temos um dado (código do produto), para essa situação específica, que não agrega absolutamente nada, tornando-se um dado descartável para a situação.

Podemos pensar da mesma forma sobre o número do patrimônio, que para um relatório simples, não se faz necessário. A nossa função `exibir()` retorna todos os elementos dos ativos em forma de lista, e cada elemento da lista é um ativo que possui: número patrimonial, data da última alteração, descrição e departamento, todos separados por “;”. Teremos que tratar cada linha (ativo) do nosso arquivo.

4.1 Slice de String

Vejamos como poderíamos retirar esse código durante a exibição simples dos ativos.

Para o nosso caso, vamos alterar o nosso arquivo “Inventario.py”, conforme demonstrado a seguir:

```
from Funcoes.Funcoes_Arquivos import *
inventario={}
opcao=chamarMenu()
while opcao>0 and opcao<4:
    if opcao==1:
        registrar(inventario)
    elif opcao==2:
        persistir(inventario)
    elif opcao==3:
        resultado = exhibir()
        for linha in resultado:
            print(linha[2:-1])
opcao = chamarMenu()
```

Código-fonte 9 – Realizando slice de string
Fonte: Elaborado pelo autor (2020)

As linhas em vermelho são as linhas que foram alteradas e/ou inseridas, em relação ao código que já existia. Vamos aos detalhes:

- Primeiro, atribuímos para a variável “resultado” os dados que forem retornados pela função **exibir()**, ou seja, o conteúdo do arquivo “inventario.csv” em forma de lista.
- Montamos um “for” para percorrer toda a lista, isso porque não queremos que saia o número do patrimônio em todos os ativos.
- De cada linha, iremos exibir somente do terceiro caractere [2] até o último [-1]. Repare que fizemos esse recorte no elemento da lista por meio do recurso “[2:-1]”, que se chama “*slice*” e representa um recorte em uma String. Cada elemento da lista é uma string, e, por isso, podemos fatiá-la e exibir somente o que desejamos. Toda string tem, para o seu primeiro caractere, a posição 0, logo, estamos descartando a posição 0 (número do patrimônio) e a posição 1 (o “;”), exibindo do caractere 2 (primeiro dígito da data da última atualização) até o final, que é representado pelo -1.

Logo, a primeira linha do meu arquivo está com: **1;11/11/2017;Impressora XPTO Laser;Recepção**. Mas será exibido apenas: **11/11/2017;Impressora XPTO Laser;Recepção**.

O recurso “slice” facilita muito a manipulação de strings, mas devemos tomar muito cuidado, pois, às vezes, a solução não é tão simples. Vejamos o que ocorreria se a minha primeira linha do arquivo estivesse com o seguinte conteúdo: **10;11/11/2017;Impressora XPTO Laser;Recepção**.

O que seria “fatiado” dessa linha???? Apenas o “1” e o “0”, isso significa que o “;” seria exibido. E se, em vez do “10”, tivéssemos o “100” ou o “1000”? Percebe que ficou engessado, quando disse querer um slice de [2;-1]? Na verdade, não é essa a lógica correta, o necessário é que ele retorne do primeiro caractere, após o primeiro “;” encontrado, até o final da string. Para isso, precisaremos localizar o “;”, a sua posição dentro da string, e é aí que entra o método **find()**.

4.2 Método find() e o poderoso método split()

O método find(), também utilizado por strings, permite que você encontre um caractere e/ou uma parte da string (conjunto de caracteres), retornando a posição da primeira incidência encontrada. Por exemplo: caso tenhamos a string “AmoPython” e executarmos um comando como: “AmoPython”.find(“o”), ele retornará o valor 2, pois a primeira vogal “o” se encontra na posição 2. Lembre-se de que a string começa do elemento zero. E caso ele não encontre o caractere desejado, retornará o valor “-1”. Agora, o nosso código do arquivo “Inventario.py” deverá ficar da seguinte forma:

```
from Funcoes.Funcoes_Arquivos import *
inventario={}
opcao=chamarMenu()
while opcao>0 and opcao<4:
    if opcao==1:
        registrar(inventario)
    elif opcao==2:
        persistir(inventario)
    elif opcao==3:
        resultado = exibir()
        for linha in resultado:
            print(linha[linha.find(";")+1:-1])
    opcao = chamarMenu()
```

Código-fonte 10 – Utilizando o método find()
Fonte: Elaborado pelo autor (2020)

O que está na cor vermelha, no código apresentado, representa a única alteração que fizemos. Substituímos o número “2” pela implementação do método `find()`, ele retornará a posição do primeiro “;” encontrado e, então, acrescentará +1 para que não exiba o próprio “;” na saída. Agora, se tivermos, em nosso arquivo “inventario.csv”, linhas como:

```
10;11/11/2017;Impressora XPTO Laser;Recepção
200;22/11/2017;Estação ABC 4GB RAM HD 1TB Processador X;Recepção
3;26/11/2017;Estação ABC 4GB RAM HD 1TB Processador X;Compras
```

A saída, mesmo com números de patrimônios com quantidade de dígitos distintos, será:

```
11/11/2017;Impressora XPTO Laser;Recepção
22/11/2017;Estação ABC 4GB RAM HD 1TB Processador X;Recepção
26/11/2017;Estação ABC 4GB RAM HD 1TB Processador X;Compras
```

Fácil, não é mesmo? Mas nossa saída ainda não está tão clara assim, poderíamos separar os dados: data da última atualização da descrição e do departamento. E, lógico, você deve ter pensado em uma solução como:

```
for linha in resultado:
    separacao=linha[linha.find(";")+1:-1]
    data=separacao[0:separacao.find(";")]
    separacao = separacao[separacao.find(";")+1:-1]
    descricao=separacao[0:separacao.find(";")]
    departamento=linha[linha.rfind(";")+1:-1]
    print("Data.....: ", data)
    print("Descrição....: ", descricao)
    print("Departamento.: ", departamento)
opcao = chamarMenu()
```

Código-fonte 11 – Desmembrando uma string
Fonte: Elaborado pelo autor (2020)

As linhas na cor vermelha são aquelas inseridas e/ou alteradas para que possamos pegar exatamente cada informação dentro de cada linha que é recuperada do arquivo “inventario.csv”. Primeiro, utilizamos uma variável “separacao” e colocamos, dentro dessa variável, o valor da variável “linha” de onde encontrar o “;” até o final, ou seja, descartaremos o número patrimonial. Com o conteúdo da minha primeira linha do arquivo, os valores das variáveis estariam assim, até o momento:

```
linha = 10;11/11/2017;Impressora XPTO Laser;Recepção
separacao = 11/11/2017;Impressora XPTO Laser;Recepção
```

Para extrair somente a data, aplicaremos o `find()` novamente, mas, desta vez, dentro da variável `separacao`. Repare que mudaremos a posição da qual utilizaremos o `find()`, porque não precisamos mais dele definindo o início do valor que desejamos, mas, sim, para delimitar o final do que desejamos. Observe, no valor da variável “`separacao`” apresentada, a data que desejamos, ela começará na posição zero, invariavelmente; já no término, precisaremos demarcar com o primeiro “;” que for encontrado, por isso, utilizamos para a variável `data`, o valor:

```
data = separacao[0:separacao.find(";")]
```

Dessa forma, os conteúdos das variáveis estarão assim:

```
linha = 10;11/11/2017;Impressora XPTO Laser;Recepção
separacao = 11/11/2017;Impressora XPTO Laser;Recepção
data = 11/11/2017
```

Agora, precisaremos da descrição, para isso, descartaremos a “`data`” da variável “`separacao`”, assim como descartamos, no início, o número patrimonial. Vamos recortar a nossa variável “`separacao`” por meio da linha:

```
separacao = separacao[separacao.find(";")+1:-1]
```

A partir dessa linha, as variáveis passarão a conter os seguintes valores:

```
linha = 10;11/11/2017;Impressora XPTO Laser;Recepção
separacao = Impressora XPTO Laser;Recepção
data = 11/11/2017
```

Assim, fica mais fácil recuperar a descrição que começará, invariavelmente, na posição zero (0) e terminará quando for encontrado o primeiro “;”. Isso justifica a linha:

```
descricao=separacao[0:separacao.find(";")]
```

Nessa linha, preenchemos a variável “`descricao`” com o conteúdo da variável “`separacao`”, utilizando os caracteres da posição zero até o “;”. Agora, as variáveis estão com os seguintes valores:

```
linha = 10;11/11/2017;Impressora XPTO Laser;Recepção
separacao = Impressora XPTO Laser;Recepção
data = 11/11/2017
descricao = Impressora XPTO Laser
```

Falta apenas o departamento o qual podemos perceber, olhando da direita para a esquerda, representado pelos caracteres até que se encontre o primeiro “;”

(considerando a leitura da direita para a esquerda) na variável “linha”, e para considerarmos a leitura da direita para esquerda, utilizaremos o método `rfind()` (r – right - direita). Os métodos `find()` e `rfind()` possuem o mesmo objetivo, entretanto, o primeiro faz a leitura da esquerda para a direita e o segundo da direita para a esquerda.

Assim, ficou fácil recuperar o departamento, no qual queremos os caracteres a partir do primeiro “;” (sentido da direita para esquerda) até o final, por isso, a linha:

```
departamento=linha[linha.rfind(";")+1:-1]
```

Agora, as variáveis estarão com os seguintes valores:

```
linha = 10;11/11/2017;Impressora XPTO Laser;Recepção
separacao = Impressora XPTO Laser;Recepção
data = 11/11/2017
descricao = Impressora XPTO Laser
departamento = Recepção
```

Finalmente, conseguimos fazer a devida separação e, então, exibirmos por meio dos prints, conforme o código apresenta. A saída ficou bem melhor agora, não é mesmo?

MAS... foi prático fazer tudo isso?

Imagine se tivéssemos 40 dados em vez de três (data, descrição e departamento). Seria uma loucura, não é mesmo? Por isso, no Python, nós temos o método `split()`, um verdadeiro herói. Ele quebra uma string de acordo com um conjunto de caracteres que você pode definir. Se olharmos o conteúdo da nossa variável “linha”, perceberemos que os nossos dados são separados por um “;” (daí a importância em não separar os dados com um símbolo comum qualquer, procure sempre utilizar símbolos que dificilmente existirão dentro de uma string, como: #, !, ##, entre outros). A função `split()` gerará uma lista e, em cada posição, teremos uma parte da string, de acordo com a quebra que foi proposta.

Por exemplo: `lista="AmoPython".split("o")` => teremos uma lista com três posições – na primeira posição, teremos “Am”; na segunda, teremos “Pyth”; e na terceira, “n”, uma vez que quebramos a string por meio da vogal “o”. Essa função facilitará muito o nosso trabalho, observe como o código ficará agora:

```
for linha in resultado:
    lista=linha.split(";")
```

```
print("Data.....: ", lista[1])
print("Descrição....: ", lista[2])
print("Departamento.: ", lista[3])
opcao = chamarMenu()
```

Código-fonte 12 – Desmembrando uma string com método Split()

Fonte: Elaborado pelo autor (2020)

Compare com a solução anterior e veja o quão simples foi essa solução proposta. Apesar de as duas funcionarem e retornarem as informações desejadas, a segunda proposta, com o método `split()`, é muito mais prática e rápida de ser implementada. Sempre que manipular Strings, terá várias formas de chegar ao mesmo resultado, mas, quando achar que o seu código está muito extenso e repetitivo, procure saber se não existe algum método capaz de simplificar o que você deseja realizar.

5 TECNOLOGIA LIGADA À SEGURANÇA DE INFORMAÇÃO

5.1 Criptografia

A criptografia ou método de escrita oculta é uma tecnologia que está diretamente ligada à segurança da informação. A partir de um conjunto de técnicas é possível criptografar arquivos ou mesmo mensagens de forma com que terceiros não consigam obter a mensagem original, assim nós conseguimos cumprir alguns critérios que são elencados dentro da área de segurança de informação como a confidencialidade.

Uma informação enviada em formato de texto claro (plaintext) pode ser lida por qualquer pessoa que tenha acesso a ela. No entanto, as mensagens criptografadas dependem de um sistema de chaves, que devem ser utilizadas para criptografar a mensagem original. Do mesmo modo, a chave precisa ser aplicada para permitir a leitura de uma mensagem cifrada.

O processo de criptografia depende diretamente de dois fatores. O primeiro é a chave, um valor secreto que vai ser usado para encriptar a mensagem, cada chave possui uma sequência própria de caracteres que quando aplicada a mensagem original a distorce, gerando um valor difícil de ser lido.

O segundo fator é o algoritmo de criptografia que vai ser usado, este comumente apontado como uma das principais falhas de segurança, uma vez quebrado, fica muito fácil descobrir qual a senha que foi utilizada para criptografar ou mesmo reverter a criptografia sem precisar de uma senha. O uso de algoritmos fortes está diretamente ligado à dificuldade que o usuário vai ter de reverter um arquivo ao seu formato original, ele pode dificultar até mesmo os ataques de brute force que precisaram rodar por muito mais tempo para conseguir chegar à mensagem original.

5.2 Data Encryption Standard (DES)

O **Data Encryption Standard (DES)** foi durante muito tempo um dos principais algoritmos para criptografia, tendo sido selecionado em 1976 como algoritmo oficial no padrão FIPS (*Federal Information Processing Standard*). Um algoritmo é capaz de gerar um embaralhamento muito forte dentro do arquivo criptografado, isso mesmo utilizando chaves de tamanho muito pequeno.

O algoritmo é, no entanto, considerado um inseguro para uso em aplicações nos dias de hoje, visto o que já em 1999 empresas foram capazes de quebrar uma chave DES. Métodos analíticos atuais também demonstram fragilidade no próprio algoritmo, sendo hoje desencorajado até mesmo o uso do 3DES que seria a forma mais segura do algoritmo.

Apesar das suas falhas de segurança, o DES ainda é muito utilizado em âmbito acadêmico para demonstrar o desenvolvimento de software de criptografia. Desta forma, o primeiro código que nós vamos gerar será justamente utilizando esse algoritmo devido à sua simplicidade. Para isso, no entanto, precisamos instalar a biblioteca PyCrypto que nos permite utilizar métodos de criptografia. Esta biblioteca não vem nativamente no Python, por isso precisamos rodar o comando “pip install pycrypto” para adicioná-la em nosso código.

Após a sua instalação, nós podemos prosseguir com o uso do código apresentado abaixo. Esse código, após importar a biblioteca, define uma chave que realiza o processo de criptografia mandar mensagem.

```
from Crypto.Cipher import DES
chave = "FIAP2020"
des = DES.new(chave, DES.MODE_ECB)
```



```
texto_claro = "LET'S ROCK THE FUTURE!!!"
texto_cifrado = des.encrypt(texto_claro)
print("Texto cifrado", texto_cifrado)
print("Texto decifrado", des.decrypt(texto_cifrado))
```

Código-fonte 13 – Criptografia com DES

Fonte: Elaborado pelo autor (2020)

Após a definição da chave, o código apresenta a chamada da função `DES.new()`, na qual será criado um modo para a criptografia DES. Este método exige que passemos dois parâmetros, sendo o primeiro a chave de criptografia e o segundo o modo de criptografia ser utilizado, nesse caso, optamos pelo uso do Electronic Code Book, este modo garante que sempre que entrarmos com a mesma chave e o mesmo conteúdo de texto, o arquivo gerado vai ser o mesmo. O próximo passo é inserimos em uma variável o conteúdo a ser criptografado, este pode ser uma string ou mesmo o conteúdo de um arquivo, a partir deste arquivo podemos gerar uma variável que conterá o conteúdo criptografado. Para isso, podemos usar um método “encrypt” que aplica a criptografia DES na variável informada. Aqui, no entanto, vale ressaltar que o texto gerado a ser criptografado deve conter número de caracteres múltiplo do número de caracteres da chave.

5.3 Advanced Encryption Standard (AES)

AES (Advanced Encryption Standard) é um algoritmo de criptografia introduzido em 2001 pelo **NIST (Instituto Nacional de Padrões e Tecnologia dos EUA)**. O algoritmo foi inicialmente desenvolvido como parte da família Rijndael de criptografia, sendo submetida e posteriormente aprovada como substituta do DES.

O AES é composto por 3 blocos com diferentes comprimentos de chave: primeiro de 128 bits, o segundo de 192 bit e o terceiro de 256 bits. Atualmente, ele está contido na maioria das bibliotecas de criptografia disponíveis para as linguagens de programação. Isso facilita muito o uso dessa criptografia nos códigos que nós podemos desenvolver. Um evento de como implementar AES pode ser visto abaixo.

```
from Crypto.Cipher import AES

chave = "FIAP2020fiap2020"
aes = AES.new(chave, AES.MODE_ECB)
texto_claro = "!!!!!!LET'S ROCK THE FUTURE!!!!!!"
```

```
texto_cifrado = aes.encrypt(texto_claro)
print("Texto cifrado", texto_cifrado)
print("Texto decifrado", aes.decrypt(texto_cifrado))
```

Código-fonte 14 – Criptografia com AES
Fonte: Elaborado pelo autor (2020)

O código é bem similar ao que nós já utilizamos anteriormente no DES, porém a chave teve que ser alterada para atender os requisitos mínimos de tamanho de chave de 16 caracteres, bem como a mensagem que deve conter um número de caracteres múltiplo de 16 para este caso. Além disso, a função também é alterada, não mais importando o conjunto DES para agora utilizar o conjunto AES da biblioteca. O restante do código, no entanto, segue igual.

6 CONVERSÃO BASE64

Apesar da versatilidade de podermos criptografar mensagens, é muito comum termos a necessidade de colocar outros tipos de conteúdo dentro de uma criptografia, sejam eles imagens, vídeos ou qualquer outro tipo de arquivo que porventura nós queiramos manter seguros.

A Base64 é um método de codificação de dados que permite converter o binário dos arquivos em uma string. Tendo este recurso amplamente utilizado por vários tipos de aplicação para permitir o envio de arquivos, esse método também é comumente chamado de codificação MIME e é constituído por 64 caracteres ([A-Z],[a-z],[0-9], "/" e "+") que estarão contidos dentro da string. O código abaixo exemplifica o uso do método base64 na conversão de um arquivo de teste.

```
import base64

with open('fiap.jpg', 'rb') as arquivo:
    binario_arquivo = arquivo.read()
    dados_base64 = base64.b64encode(binario_arquivo)
    mensagem_base64 = dados_base64.decode('utf-8')

    print(mensagem_base64)
```

Código-fonte 15 – Conversão Base64
Fonte: Elaborado pelo autor (2020)

O código acima abre uma imagem intitulada "fiap.jpg" através do modo de leitura binário, em seguida, o conteúdo binário da imagem é passado para uma variável, e essa passa pelo processo de codificação base64. O código possui ainda

uma linha com “decode”, essa linha vai gerar uma mensagem para facilitar o uso do conteúdo, uma vez que só a conversão do dado em base64 vai deixar um “b” na mensagem, o que depois teria que ser tratado para poder fazer a desconversão, por fim, o código ainda printa o conteúdo da figura convertida.

7 SIMULANDO UM ATAQUE RANSOMWARE

7.1 Finalizando o capítulo

Para finalizar, você poderá aplicar todos os conceitos vistos desenvolvendo uma ferramenta capaz de simular um ataque de *ransomware*. Este ataque ficou famoso nos últimos anos após a sua popularização pelo o WannaCry.

Um ataque *ransomware* consiste em abrir um arquivo dentro do código, criptografá-lo e remover o arquivo original deixando apenas o arquivo criptografado, o que impede que o usuário faça uso deste. Formas mais avançadas desse ataque utilizam ainda outros tipos de criptografia, como a assimétrica, isso dificulta a tentativa de abertura do arquivo através de bruteforce, dado o tamanho da chave. Uma vez que o arquivo é fechado com uma chave pública, apenas o atacante será capaz de descriptografá-lo utilizando a sua chave privada.

Agora, você consegue armazenar os seus arquivos de forma segura e também simular ataques. Sim, isso mesmo! Você será capaz de realizar ataques de ransomware nos ambientes que criar durante o curso. Isso é fundamental para permitir que você conheça os fundamentos por trás desses tipos de ataque. No entanto, o uso de tais técnicas exige grandes responsabilidades, por ora aproveitaremos para utilizar os novos aprendizados.

REFERÊNCIAS

FORBELLONE, A. L. V. **Lógica de programação**. 3. ed. São Paulo: Prentice Hall, 2005.

KUROSE, J. F. **Redes de computadores e a Internet**: uma abordagem top-down. 6. ed. São Paulo: Pearson Education do Brasil, 2013.

MAXIMIANO, A. C. A. **Empreendedorismo**. São Paulo: Pearson Prentice Hall Brasil, 2012.

PIVA JÚNIOR, D. **Algoritmos e programação de computadores**. São Paulo: Elsevier, 2012.

PUGA, S. **Lógica de programação e estruturas de dados**. São Paulo: Prentice Hall, 2003.

RAMEZ, E.; SHAMKANT, B. N. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

RHODES, B. **Programação de redes com Python**. São Paulo: Novatec, 2015.

STALLINGS, W. **Arquitetura e organização de computadores**. 8. ed. São Paulo: Prentice Hall Brasil, 2010.

VISUAL STUDIO CODE. [s.d.]. Disponível em: <https://code.visualstudio.com>. Acesso em: 12 fev. 2020.